



Program: BS Computer Science

Semester: Fall 2025

Credit Hour(s): 1

Pre-requisite(s): EE1005 Digital Logic Design

Instructor: Engr. Muhammad Hassaan Bin Shoukat – hassaan.shoukat@nu.edu.pk

Lab # 3: Addressing Modes - I

1. Lab Objectives

- Learn how to declare data in memory using directives.
- Understand how to access data directly using memory addresses.
- Identify and fix size mismatch errors when working with registers and memory.

2. Introduction

In previous labs, we worked with registers and simple instructions. Now, we move one step further and learn how to use **memory locations**, means that the ways to tell CPU where to find data.

To use memory properly, we first need to **declare data**, then learn how to **access** it. We'll also understand size mismatch errors that happen when we confuse the assembler about whether we want a *byte* or a *word*.

3. Data Declaration in Assembly

Before using variables or labels, we must declare them.

Syntax: `variable/label_name size_directive value(s)`

Common directives:

- **DB** - Define Byte (8 bits – 1 byte): Use this for small numbers and characters.
- **DW** - Define Word (16 bits – 2 bytes): Use this for larger numbers and memory addresses.

Examples:

- `num1 db 25`
- `num2 dw 300`
- `array db 10,20,30,40,50`
- `message dw 'Hello', 0`

Remember:

- A **byte variable** can store values from 0 to 255 (unsigned) or -128 to +127 (signed).
- A **word variable** can store values from 0 to 65535 (unsigned) or -32768 to +32767 (signed).



4. Direct Addressing

In direct addressing, the instruction **directly specifies the memory location** of the data. Direct addressing uses a variable's label (its memory address) to access its value.

The syntax is:

```
MOV destination, [address]
MOV [address], source
```

The Brackets Rule:

- *myByte* is the **address** (like a house number).
- *[myByte]* is the **value at that address** (like the people inside the house).

Example:

```
MOV AL, [2000H] ;loads the byte at memory address 2000H into AL
MOV [2000H], BL ;stores the value in BL into memory address 2000H
```

5. Size Mismatch Errors

The CPU is very strict about data sizes. Sometimes, the assembler doesn't know whether you want to work with a **byte** or a **word**. You cannot fit a 2-byte value into a 1-byte space. NASM will catch this when you try to assemble the code, and it will give an error. This specific error is called **size mismatch error**.

Incorrect Example:

- `mov ax, [2000H]` ; Assembler doesn't know if [2000H] is byte or word
- `my_word dw 1000` ; my_word is 16 bits
- `mov bl, [my_word]` ; BL is 8 bits

6. Examples

Example 1: Individual Word Variables (No Offset)

```
[org 0x0100]
mov ax, [val1]
mov bx, [val2]
add ax, bx
mov cx, [val3]
add ax, cx
mov [result], ax
mov ax, 0x4c00
int 0x21

val1: dw 7
val2: dw 14
val3: dw 21
result: dw 0
```



Example 2: Word Array Elements Using Memory Offsets (Single Label)

```
[org 0x0100]

    mov ax, [value]
    mov bx, [value + 2]
    add ax, bx
    mov cx, [value + 4]
    add ax, cx
    mov [value + 6], ax

mov ax, 0x4c00
int 0x21

value: dw 8
       dw 12
       dw 20
       dw 0
```

Example 3: Word Array Summation with Offsets (Single Label)

```
[org 0x0100]

    mov ax, [data]
    mov bx, [data + 2]
    add ax, bx

    mov cx, [data + 4]
    add ax, cx

    mov [data + 6], ax
mov ax, 0x4c00
int 0x21

data: dw 6, 9, 12, 0
```

Example 4: Direct Memory Accumulation (Single Label)

```
[org 0x0100]

    mov ax, [val]
    mov [val + 6], ax

    mov ax, [val + 2]
    add [val + 6], ax

    mov ax, [val + 4]
    add [val + 6], ax
mov ax, 0x4c00
int 0x21

val: dw 7, 14, 21, 0
```



Example 5: Byte Array Elements Using Offsets

```
[org 0x0100]
```

```
    mov ax, [arr]
    mov bx, [arr + 1]
    add ax, bx

    mov bx, [arr + 2]
    add ax, bx

    mov [arr + 3], ax

mov ax, 0x4c00
int 0x21

arr db 4, 8, 12, 0
```

7. Lab Tasks

Task 1: Write a program that defines the following:

- A byte named score initialized to 100.
- A word named population initialized to a hexadecimal value 2500.
- A byte named initial initialized to the character 'F'.
- An array of 4 bytes named numbers initialized to 10, 20, 30, 40.

Compare and mention their memory usage in the debugger and represent in a tabular form.

Task 2: Write code to:

1. Copy the value of score into the BL register.
2. Copy the value of population into the CX register.
3. Copy the value of initial into the DH register.
4. Copy the value from the AX register into the population variable.
5. Copy the value from the CL register into the score variable.

Task 3: Write a program with a size mismatch error by moving a DW value into an 8-bit register. Assemble, record the error, and fix it. Add comments explaining the mistake and solution.

Task 4: The following code snippets have errors. For each line, identify the error and write the correct statement.

```
    mov [var1], 99h
    mov ax, [var2]
    mov bl, var1

    var1 dw 1234h
    var2 db 12h
```